Team 20 Hackathon Report

Querying Strategy

The goal of our querying strategies was to balance exploitation, exploration, and redundancy control. To maximize our 3 query limit, we implemented strict constraints at each round to prevent repeat-querying mutants. The first round of querying was focused on exploitation and exploration. Using a HYBRID_NN_SCORE, equal weight was given to the predicted fitness and to the distance from the labelled set we were provided with. This allowed for selection of potentially more valuable regions that were dissimilar from the labelled set. Hitting 51 unique positions (notably favoring positions 651 and 653), this query strategy yielded positive results with the mean DMS score being 0.778 and median being 0.841, indicating high fitness.

For the second round of querying, we took a multi-strategy portfolio approach, combining Upper Confidence Bound/Estimated Improvement (UCB/EI), GREEDY, HYBRID, and RANDOM to capture different aspects of the protein space. The strategies were given the budgets 0.28, 0.28, 0.24, 0.20, respectively. We also implemented redundancy controls in this querying round. Diversity penalties were applied to UCB/EI selections to prevent selecting mutants that were too similar to one another. Destination constraints were used to balance amino acid representation in the selections. Diagnostic steering of the model was developed from errors in the first querying round, focusing on substitution type errors. This approach improved the model from the first querying round, increasing the unique positions to 61, none of which overlapped with round one, and producing improved fitness: a mean DMS score of 0.801.

The final querying round was focused on the fine tuning of the previous two rounds, i.e., improving on weaknesses and remaining failures. Our priority was labelling the mutants the model predicted as the best in an attempt to correctly identify the true fitness of the high-prediction tail as possible to submit a robust list of the top 10 mutants. To do this, we used a tail-first portfolio: a small neighborhood prefill around our current top-10 candidates in embedding space, followed by GREEDY selection of the highest-predicted mutants, a UCB slice with diversity penalties to avoid near-duplicate queries, a smaller HYBRID hedge toward unlabeled regions, and a small RANDOM remainder for calibration, while also enforcing destination amino acid caps/floors and turning off diagnostic weighting so the budget stayed concentrated on the top-prediction tail.

Model Architecture and Justification

Architecture Selection Rationale: The primary objective of the hackathon was to predict a continuous fitness score, or Deep Mutational Scanning (DMS) score, for single amino acid substitutions. The selection of our model architecture was driven by three core constraints:

1. High-Dimensional Inputs vs. Small Labeled Data: The dataset contained hundreds of labeled mutants but thousands of Protein Language Model (PLM) embedding dimensions. Utilizing a single end-to-end deep neural network was rejected because it would easily memorize the training set, creating a severe overfitting risk .

2. Heterogeneous Signal Sources: The inputs included biophysical descriptors, evolutionary priors, and deep sequence context. Because these carry complementary information, they are best captured by different model families.

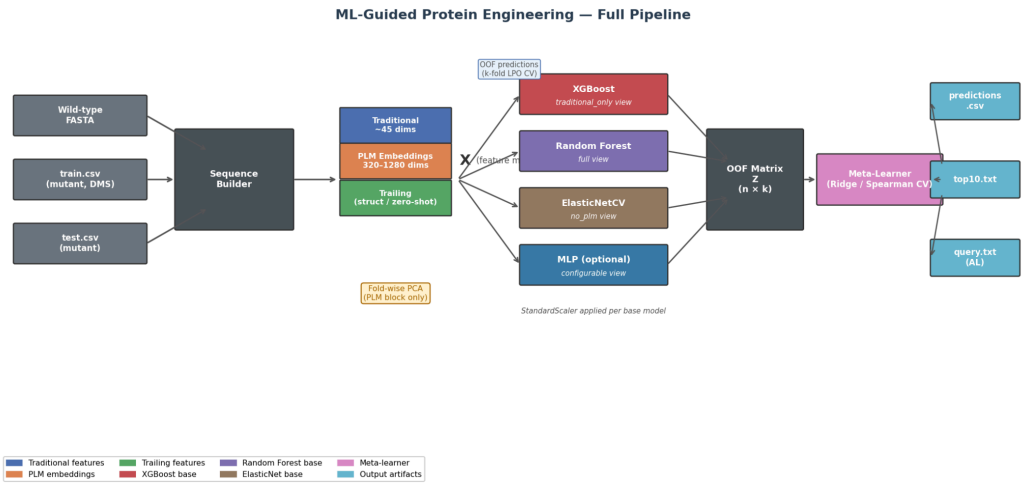
3. Ranking-Based Objective: The final performance was evaluated on Spearman correlation, meaning the model needed to correctly order mutants rather than strictly minimize mean squared error (MSE).

Based on these 3 constraints, we implemented a two-level stacked ensemble architecture. This allows diverse base models to operate on specialized feature views matched to their inductive biases, with their out-of-fold (OOF) predictions synthesized by a Spearman-tuned meta-learner.

Full Pipeline Overview:

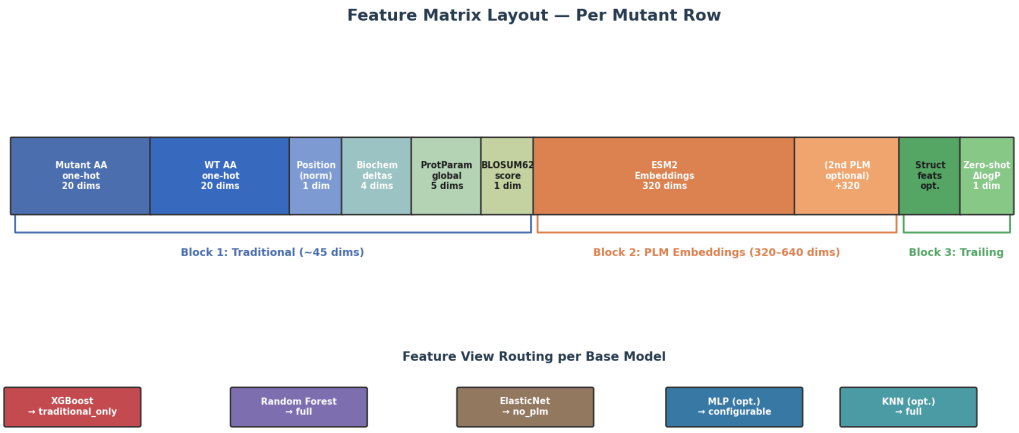
Inputs: The Sequence Builder ingests the wild-type FASTA alongside train.csv (labeled mutants) and test.csv (unlabeled mutants)

- Feature Construction: It constructs a concatenated feature matrix composed of Traditional, PLM Embedding, and Trailing blocks
- Routing & Stacking: Data is strategically routed to distinct Level 1 base models (XGBoost, Random Forest, ElasticNetCV, and an optional MLP) based on their inductive biases
- Fusion & Output: The out-of-fold (OOF) predictions from these base models are fused by a Meta-Learner to produce the final artifacts: predictions.csv, top10.txt, and active learning queries (query.txt)



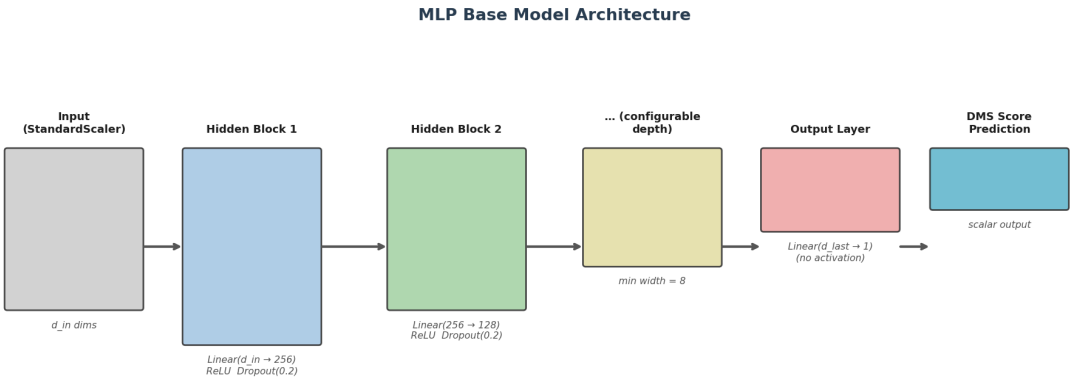
Feature Architecture and Routing: The feature matrix is divided into specific blocks, allowing each base model to be routed to its designated slice without recomputing features

- Block 1: Traditional Engineered Features (~45 dimensions): Includes one-hot encoded wild-type and destination amino acids (20 dimensions each), normalized mutation position, ProtParam global sequence descriptors (e.g., molecular weight, isoelectric point), biochemical mutation deltas (e.g., hydrophobicity, charge), and normalized BLOSUM62 substitution scores
- Block 2: PLM Embeddings (320–1280+ dimensions): Contains primary embeddings extracted from ESM2 checkpoints (default: ESM2-8M) via mean-pooling at the final transformer layer
- Block 3: Trailing Features: Supports structural features (e.g., SASA, B-factors) and a zero-shot pseudo-log-likelihood score via masked ESM2 inference.



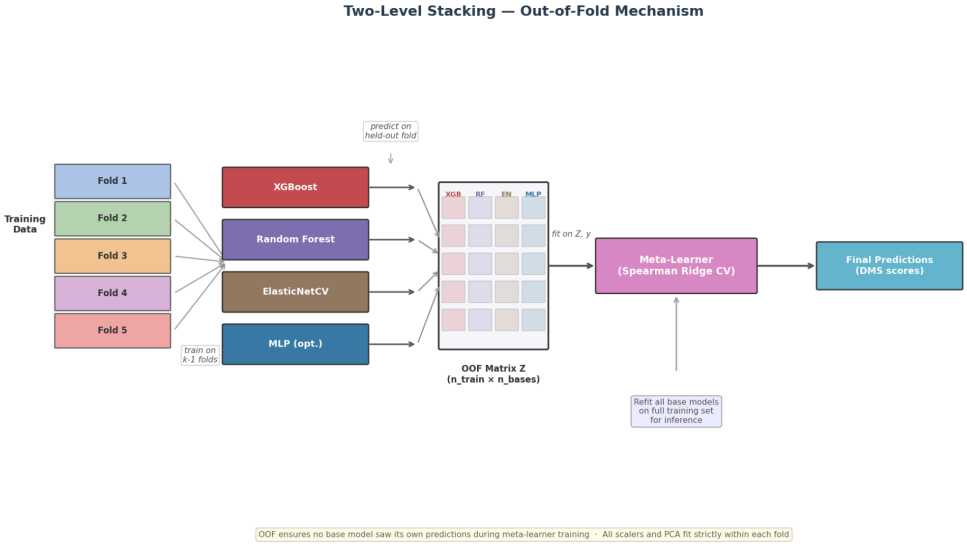
Base Model Architecture

- XGBoost Regressor: Routed exclusively to the traditional feature view.
    - Rationale: Gradient boosting captures non-linear interaction effects among compact biophysical descriptors without being drowned out by high-dimensional PLM embeddings.
    - Loss/Objective: Uses a standard squared-error objective, but hyperparameters (tuned via Optuna TPE) are selected based on cross-validation (CV) Spearman correlation.
- Random Forest Regressor: Trained on the full concatenated feature set.
    - Rationale: Benefits from implicit feature selection, allowing it to average out noisy PLM dimensions while leveraging all available data.
    - Loss/Objective: Standard squared-error objective with Spearman-targeted CV selection
- ElasticNetCV: Trained strictly on the no-PLM view (Traditional + Structural features)
    - Rationale: Provides a strong, interpretable linear baseline that prevents overfitting to spurious non-linearities in the small dataset . PLM embeddings are deliberately excluded because exposing a linear model to 320+ correlated dimensions leads to ill-conditioned and degenerate solutions.
    - Architecture/Hyperparameters: StandardScaler-normalized inputs . It searches an alpha grid of 20 log-spaced values from 1e-4 to 1.0, and an l1_ratio grid of [0.1, 0.5, 0.9] to blend L1 and L2 regularization . It utilizes 3 inner CV folds and a maximum of 8000 coordinate descent iterations to ensure convergence . It optimizes L1+L2 penalized MSE
- MLP Regressor: Input data is passed through a StandardScaler into Hidden Block 1 (Linear layer reducing inputs to 256 dimensions, followed by a ReLU activation function and Dropout at 0.2) . This is followed by a configurable number of hidden blocks, terminating in an Output layer (Linear mapping to 1 dimension) with no activation function, yielding a scalar DMS prediction




```
                    Training details:
                Loss:       MSE  (mean squared error)
             Optimizer:   AdamW  (lr=1e-3,  weight_decay=1e-4)
Epochs:      120 max   |   Patience: 20  (early stopping on val MSE or Spearman)
Batch size:  64        |   Optional Mixup augmentation  (λ ~ Uniform[0.5, 1.0])
        Scaler:      StandardScaler fit strictly on training fold only
```

Two-Level Stacking and Out-of-Fold (OOF) Mechanism: The out-of-fold (OOF) predictions from all base models are collected to form an OOF matrix, which is then passed to a Ridge-style meta-learner. The fusion step is explicitly tuned in an inner cross-validation loop to directly maximize Spearman correlation, aligning the meta-learner with the competition evaluation metric rather than minimizing MSE.



Task-Specific Modifications and Regularization: To ensure robustness and avoid data leakage, several key pipeline modifications were implemented:

- Leave-Position-Out (LPO) Cross-Validation: Instead of random splits, CV folds were grouped by mutation position. This accurately simulates extrapolation to unseen sequence positions and prevents position-level data leakage .

- Fold-wise PCA: Principal Component Analysis on the high-dimensional PLM block was fit strictly inside each training fold and then applied to the validation fold, preventing validation data from influencing the dimensionality reduction
- SMOTE Tail Oversampling: Applied to the top 10% of the heavily right-skewed DMS score distribution to focus model capacity on the competitively important, rare high-fitness mutants
- Embedding Noise Injection: Gaussian noise was added to PLM embedding columns during tree-learner training to reduce over-reliance on any single embedding dimension.
- Repeated Holdout Evaluation: Models were evaluated and promoted based on the mean Spearman correlation across 5 independent holdout draws (each holding out ~12% of the data), guarding against selection bias from a single lucky split

Training Details

Data Preprocessing: We utilized our labeled samples in CSV format, obtained as output at each active learning stage, and concatenated them into one file. We kept a 12% holdout, set aside randomly (and saved), right at the beginning, such that no matter how many times we ran this experiment, the same samples remained out of the training dataset.

Feature Engineering: Each mutant was encoded using three types of feature vectors combined into one matrix:

- Biochemically engineered features (about 50 dimensions): One-hot vectors for mutant and wild type amino acid identities, normalized residue positions, Kyte-Doolittle hydrophobicity change, charge difference, and sequence statistics (mass, pI, aromaticity, instability index, and GRAVY score) calculated using BioPython. Additionally, BLOSUM62 substitution score was optionally provided.
- PLM embeddings: Extracted by averaging token representations generated by a frozen protein language model. Supported models were ESM-2 (with parameter counts ranging from 8M to 650M), ESM-1v, and ProtT5-XL. The embeddings were stored in memory to prevent inference at each run. Optionally, another model's embeddings could be concatenated.
- Zero-shot scores: Log likelihood differences (log P(mutant) - log P(wild-type)) produced by ESM-2 masked LM head.

Dataset Splitting: The split ratio for our model was 12% to be held out for final testing, while we trained using the remaining data. The training dataset was further divided using Leave-Position-Out cross-validation where the mutants were grouped based on their position of mutation, enabling the construction of validation sets consisting only of positions that were never encountered during the training process.

Model Training: Our solution is based on a stacking framework, whereby several base models were trained (namely XGBoost, Random Forest, Ridge Regression, Elastic Net, K-Nearest Neighbors, and a small MLP), which created out-of-fold predictions that served as inputs to the meta-model (RidgeCV by default). Non-ensemble models utilized StandardScaler scaling before training. The MLP base consisted

of two 256-neurons layers with ReLU activation, Dropout of 0.2, and AdamW optimizer with early stopping.

Hyperparameter Tuning: Optuna with TPE sampling was employed for automatic hyperparameters tuning. We optimized XGBoost parameters: max_depth, learning_rate, subsample, colsample_bytree, reg_lambda, and n_estimators for 20 iterations. We optimized hidden_layer_size and drop probability for MLP for 15 iterations. By default, hyperparameter tuning was not enabled due to its high computational cost.

Addressing Overfit: Overfitting was a problem since we had very little data labeled at any given time. Overfitting was tackled by: fold-sensitive PCA on the PLM embeddings (trained only using training folds in order to prevent information leak), L2/elastic-net regularization for linear models, dropout & early stopping for MLPs, adding embedding noise for regularization, and using LPO cross-validation to obtain unbiased validation scores.

Challenges: A major challenge for the project was the lack of data: with 100 newly labeled data points in each iteration, the model was learning from a very small growing dataset. Hence, using PLM embeddings posed an overfitting risk due to their large feature space compared to the training data size, requiring us to properly regularize and conduct PCA. Finally, one more problem was the runtime, since PLM-based inference is relatively slow, requiring us to cache embeddings based on certain parameters.

Model Performance Analysis

In addition to our leaderboard evaluation, we used multiple other metrics to assess our model's effectiveness. Early on, we determined that the rubric heavily penalizes absolute error metrics in favor of ranking capability. For this reason, we treated the Spearman correlation as the ultimate ground-truth metric for all model selection. Mean squared error and root mean squared error were treated strictly as secondary diagnostics since they are more absolute.

For local testing, we evaluated using a 5-repeat holdout strategy. The 5-repeat holdout strategy randomly partitions the dataset for training and testing, trains the model, evaluates performance on the test data, and then repeats the process a total of five times with different random splits. The final performance is evaluated using an average score across the five splits. This ensured that we selected architectures that won the mean distribution, rather than scoring highly on a single lucky split.

The pipeline upgrades translated directly to external performance gains. Our Spearman correlation scores for each milestone were as follows:

- *Baseline Checkpoint:* ~0.05 Spearman (naive model).
- *Post-Round 1:* ~0.19 Spearman (following PLM integration and initial active learning).
- *Post-Round 2:* ~0.39 Spearman (following the implementation of the stacked ensemble, fold-wise PCA, and the portfolio active learning strategy).

Overall, we were able to follow our model's progress and accuracy throughout the course of the hackathon using these metrics.

Reflections and Challenges

Our final results included the predictions.csv and top10.txt files. Our model performed well, with a final Spearman metric of ~0.39. The drivers for our model's success include optimizing for the Spearman metric, relying on LPO cross validation for unbiased validation scores, providing heterogeneous feature views to a stacked ensemble, and enforcing controls that restrict query redundancy during active learning.

Challenges during this hackathon included lack of data, leakage traps, computational bottlenecks, and query redundancy.

- *Lack of Data:* Since there were only 100 newly labeled data points with each round of querying and a large feature space, we risked overfitting. To mitigate this, we performed regularization techniques and fold-wise PCA.
- *Leakage Traps:* Data leakage is a major concern, specifically when fitting transforms (like PCA) are used on overly broad portions of data instead of strictly within cross-validation folds. We took extreme caution to ensure that these transformations were only performed within the training folds and never on the full dataset.
- *Compute Bottlenecks:* Running large protein language models and Optuna sweeps concurrently created a risk of exhausting the available RAM and compute times. We took extra care with caching and curating our job constraints in order to prevent possible bottlenecks from occurring.
- *Query Redundancy:* If we did not limit redundancies, we would waste a portion of our limited querying budget. By implementing diversity penalties, we were able to ensure that mutants were not too similar to each other during the querying stages.

Potential improvements could be made if we had additional time and computing resources. Our pipeline would benefit from rank-aware loss functions for the neural networks, principled pruning of the base models in the stack, and applying Parameter-Efficient Fine-Tuning (PEFT/LoRA) to the ESM models rather than keeping them frozen.

Contributions

We initially started with a "divide and conquer approach", agreeing to keep building on whoever's model performed the best by the first checkpoint. From there on, contributions were approximately equal across the board.

Codebase

https://drive.google.com/drive/folders/1VzlG9DluRH5zaWPPwxfoA_5kDGUEK7s8?usp=sharing